Set up Qase and GitHub

For a new audience: prepare access and reporting once. The existing Triangle project already has the apps and tests.

1 Sign in and choose a Qase project

Create a Qase account and workspace, or accept a workspace invitation. Open Projects. Reuse Triangle, code TR, or create a project and record its code. You need permission to create cases and runs.

Project = the product. Suite = a group of cases. Case = inputs, steps and expected result. Run = one execution with results.

2 Activate the reporter apps and create tokens

Open Qase Apps. Open Vitest and activate it if needed. In Access tokens, create a token. Repeat for Playwright. Keep the two tokens separate so their results appear under the corresponding app.

3 Use the existing GitHub workflow

Open tiara-stack/triangle, or fork it to your own account and enable Actions. You need permission to run workflows and configure repository secrets and variables.

4 Add the repository variable and two secrets

In your GitHub repository, open Settings > Secrets and variables > Actions. Use the Variables tab for the project code and the Secrets tab for the two tokens.

GitHub setting	Name	Value
Repository variable	QASE_TESTOPS_PROJECT	TR, or your project code
Repository secret	QASE_VITEST_API_TOKEN	Vitest app token
Repository secret	QASE_PLAYWRIGHT_API_TOKEN	Playwright app token

A shared personal token can instead use the single secret QASE_TESTOPS_API_TOKEN. For separate app tokens, use the two secret names above. Never paste a real token into a case, slide, or repository file.

> **Check the connection before presenting**
>
> Launch Performative defect detection once. Its configuration step should say "Qase TestOps reporting is enabled." Verify four results in each new Qase run. The classifier assertions should fail in two cases per suite. Local setup commands are on page 6.

Setup references: Qase first steps / Reporter tokens / Playwright app / GitHub secrets

Test four triangle cases

Use Qase to define the expected result, record a manual run, and review automated results.

Open Triangle Lab | Open Qase project TR | Lesson 3 specification

Case	Inputs a, b, c	Expected in Qase	Actual classifier output	Record
01 a and c match	2, 1, 2	Isosceles	Scalene	**Failed**
02 200 is allowed	1, 200, 200	Isosceles	Value is out of range	**Failed**
03 Equilateral	3, 3, 3	Equilateral	Equilateral	**Passed**
04 201 is rejected	201, 1, 1	Value is out of range	Value is out of range	**Passed**

Allowed side lengths are whole numbers from 1 through 200, including 200. Three equal sides are Equilateral. A valid triangle with two equal sides is Isosceles.

2 passed + 2 failed
Expected in the manual run and in each automated suite.

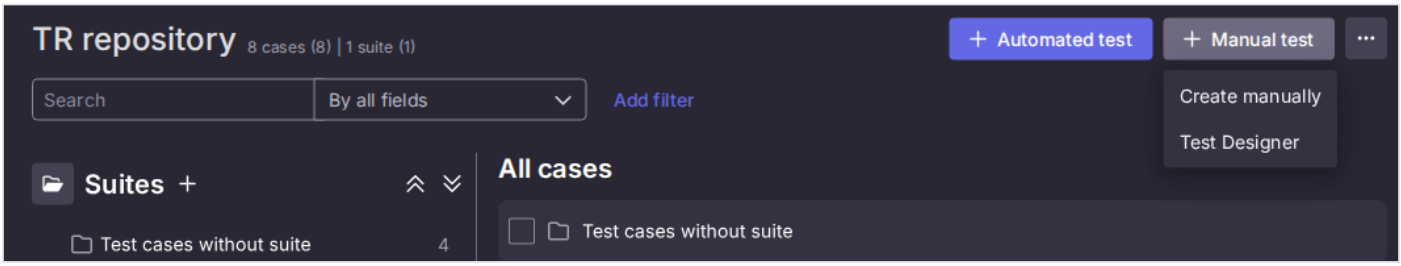


Current demo build: inputs 2, 1, 2 produce Scalene. Record that actual value in Qase and mark this case Failed.

For a short presentation, prepare the four cases first. Start the automated job, then run the manual checks while it works. A convenient manual order is 03, 01, 02, 04.

Register the four cases

Prepare these once in the Triangle project, code TR. Use four separate manual cases.



Your Triangle repository. Use Manual test, then Create manually.

1. **Open Repository and create a suite**
   Name the suite "Triangle demo - four cases".

2. **Open the full case editor**
   Use Manual test, then Create manually. Choose this suite, leave Status as Actual, choose Manual for Automation status, and set Type to Functional. Under Test Case Steps, use Classic and New step.

3. **Enter the case content and save**
   Use the example below for case 01. The other three use the same action, with their own inputs and expected result from page 2.

Example content for case 01

Title	Classifies a = c as Isosceles
Preconditions	Triangle Lab is open. Select Enter sides only.
Classic step / Action	Enter the three side lengths, click Run test, and read the classifier output.
Data	a = 2; b = 1; c = 2
Expected result	**Isosceles**

Use "Save and create another" to add the remaining cases. Suggested titles: "Accepts 200 as an allowed side length", "Classifies equal sides as Equilateral", and "Rejects a side length of 201".

> **Use the specification for Expected result**
> Qase assigns a case ID with the TR prefix. Keep those IDs. Record the faulty output in the run's Comment field.

Record a manual run

The lab shows the classifier's actual output. Your Qase case supplies the expected result.

1. **Start an express run**
   In Repository, select the four cases and click Run. Name it "Triangle demo - Manual" and start it.

2. **Open the run wizard**
   Open any case in the run, or use Open Wizard. Keep Triangle Lab open in a second tab.

3. **Execute the case in Triangle Lab**
   Select Enter sides only. Enter its a, b, c values and click Run test. Read the value under Observed / Faulty classifier.

4. **Record the result in Qase**
   Compare that value with the case's expected result. Use Passed when they match and Failed when they differ. Add a brief comment and a screenshot for each failure.

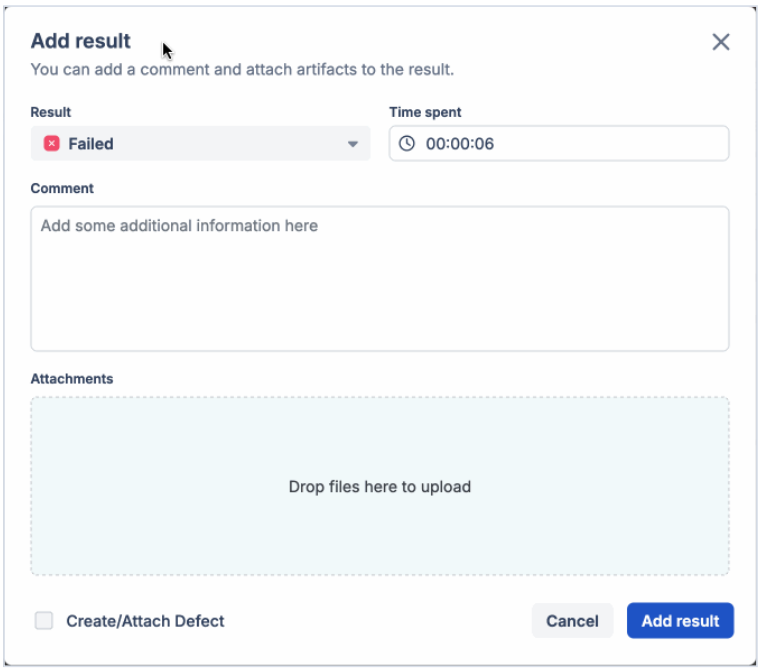
Example failure comment

Inputs: a=2, b=1, c=2.

Expected: Isosceles.
Actual: Scalene.

The classifier missed the equal a and c sides.

Choose Failed, add the comment, attach the lab screenshot, and save the result. A successful HTTP request still needs the correct classification to pass.



Qase Add result dialog. Official help example, cropped to show the result fields.

Finish the four cases and complete the run

Cases 01 and 02 should be Failed. Cases 03 and 04 should be Passed. The completed manual run should show 2 passed and 2 failed.

Trigger once, review two runs

GitHub Actions executes the current tests. The installed reporters send their results to project TR.

GitHub Actions
Run workflow on main

→

Vitest + Playwright
4 cases in each suite

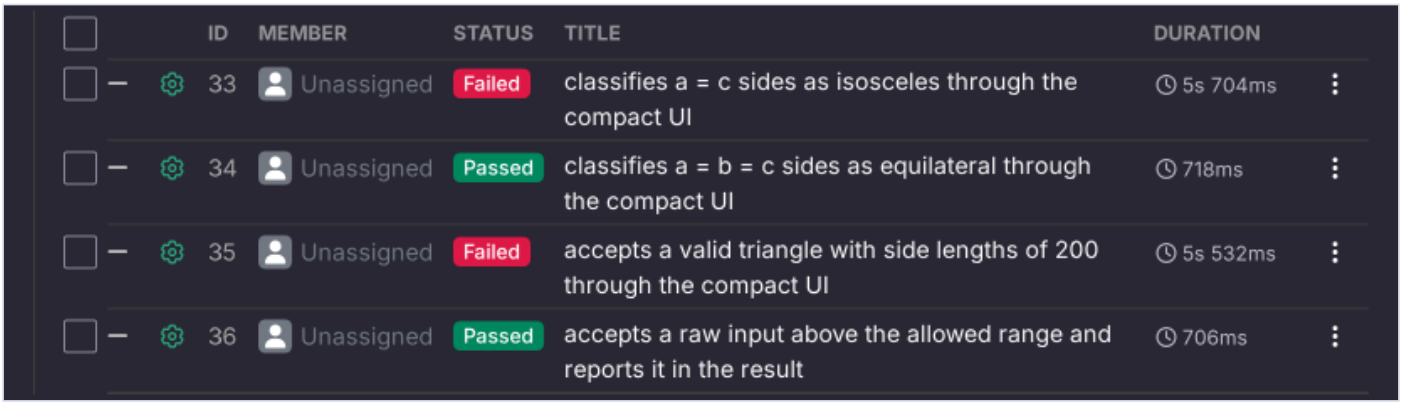
→

Qase Test Runs
Two result reports

1. **Launch the existing workflow**
   In your repository or fork, open Performative defect detection. Click Run workflow, choose main, and click Run workflow again.

2. **Confirm that the tests ran and reports uploaded**
   Open the new job. In its test step, check that failures concern the a = c and valid-200 cases, and confirm that Qase reporting is enabled and that the reporters uploaded results. These assertion failures make the workflow red.

3. **Inspect the newest Qase results**
   In Triangle / Test Runs, open the newest runs ending in "Direct Effect" and "Browser UI". Open a failed result to show its assertion details. Each run should contain four checks. If Qase still says In Progress after all four have results, use Complete to close the run.



ID	MEMBER	STATUS	TITLE	DURATION
33	Unassigned	Failed	classifies a = c sides as isosceles through the compact UI	5s 704ms
34	Unassigned	Passed	classifies a = b = c sides as equilateral through the compact UI	718ms
35	Unassigned	Failed	accepts a valid triangle with side lengths of 200 through the compact UI	5s 532ms
36	Unassigned	Passed	accepts a raw input above the allowed range and reports it in the result	706ms

Actual Browser UI report: 2 passed and 2 failed. Direct Effect report / Browser UI report

These totals were verified in workflow run 40702371101 at revision 785557b. A new launch produces new run IDs. Across the two runs, the four scenarios appear twice, for eight results total.

Case IDs and shared history

The current code has no explicit Qase case IDs. Automated results match by title and suite path, so they can appear separately from the manual cases you created. Link the real Qase IDs in both test files to share one case history. If first-run failing cases are absent, enable case auto-creation for all statuses or link IDs.

References: Qase cases / Qase runs / Linking tests / GitHub manual trigger
Image credit: Qase Help Center, GitHub integration. Other Qase screenshots are from project TR. Lab image is from the current demo build.

Run the same tests locally

Use this when following along on your own machine. Manual testing only needs a browser and Qase.

1 Install the tools and get the repository

Use Git, Node.js 22.18 and pnpm 11.9.0 to match this repository's workflow. Clone the repository or your fork, then install its locked dependencies.

```
git clone https://github.com/tiara-stack/triangle.git
cd triangle
pnpm install --frozen-lockfile
node node_modules/@playwright/test/cli.js install chromium
```

On Linux, Chromium may also require OS packages. Use the Playwright install command with --with-deps chromium if the browser reports missing libraries.

2 Create .env.local for Qase reporting

Copy .env.local.example to .env.local. Fill in the project code and app tokens below. Leave QASE_TESTOPS_API_TOKEN empty when using the two app tokens. This file is ignored by Git.

```
QASE_TESTOPS_PROJECT=TR
QASE_VITEST_API_TOKEN=<your Vitest app token>
QASE_PLAYWRIGHT_API_TOKEN=<your Playwright app token>
```

3 Run both suites and publish their results

Run pnpm test:performative:qase. The script loads .env.local with Effect ConfigProvider, starts Vitest, then Playwright, and selects each reporter's token. Playwright builds and starts the local app automatically.

For a private rehearsal without uploading results, use QASE_MODE=off pnpm test:performative. A two-failure exit is expected while the two classifier defects remain.

Find the code when adapting this exercise

Direct tests: tests/performative/triangle.test.ts
UI tests: tests/e2e/performative.spec.ts
Reporter selection: vitest.node.config.ts and playwright.config.ts
Token routing and local config: scripts/performative-test.ts
GitHub trigger: .github/workflows/performative.yml

The two reporters are already installed and enabled conditionally in these configs. Shell environment variables take precedence over .env.local. Current automated cases match by title and suite path; register manual cases separately unless you add real Qase case IDs to both test files.